Chapter 11

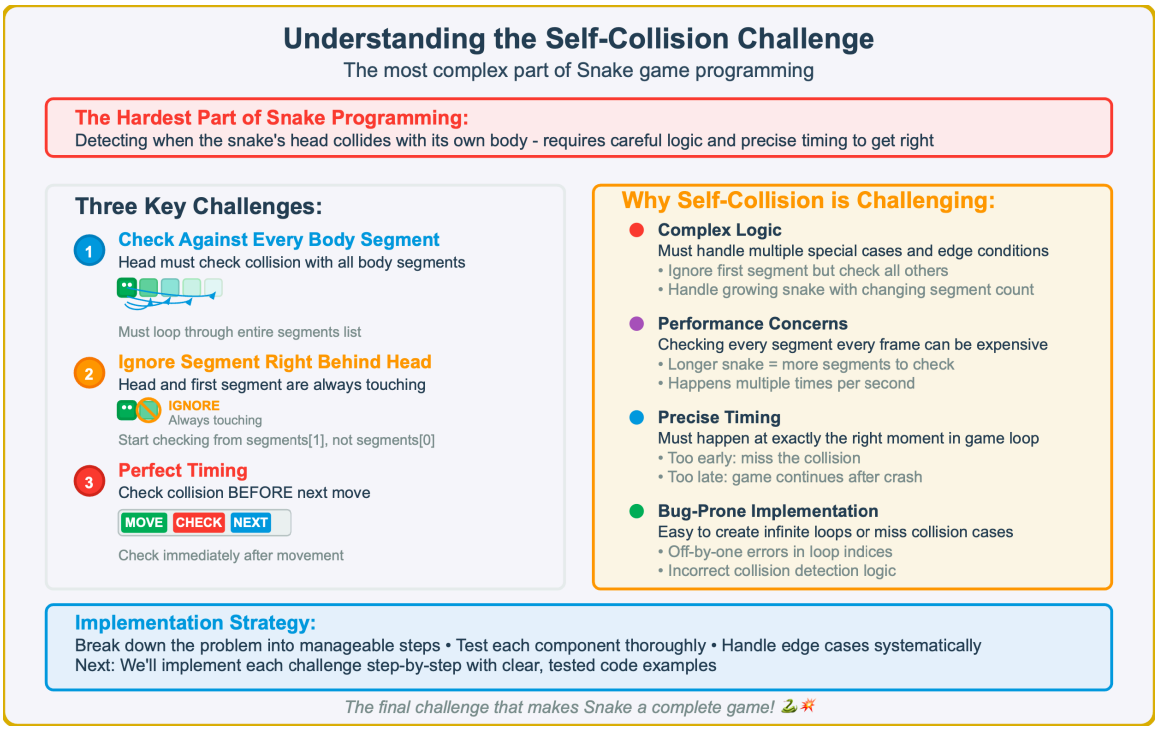

Figure 11.1 — Self—Collision Detection Challenge — The Hardest Part of Snake Programming. Self—collision detection requires solving three key challenges: (1) checking the head against every body segment by looping through the entire segments list, (2) ignoring the segment directly behind the head since head and first segment are always touching, and (3) perfect timing by checking collision immediately after movement but before the next move. This complex logic involves handling special cases, performance considerations, precise timing, and is highly bug—prone.

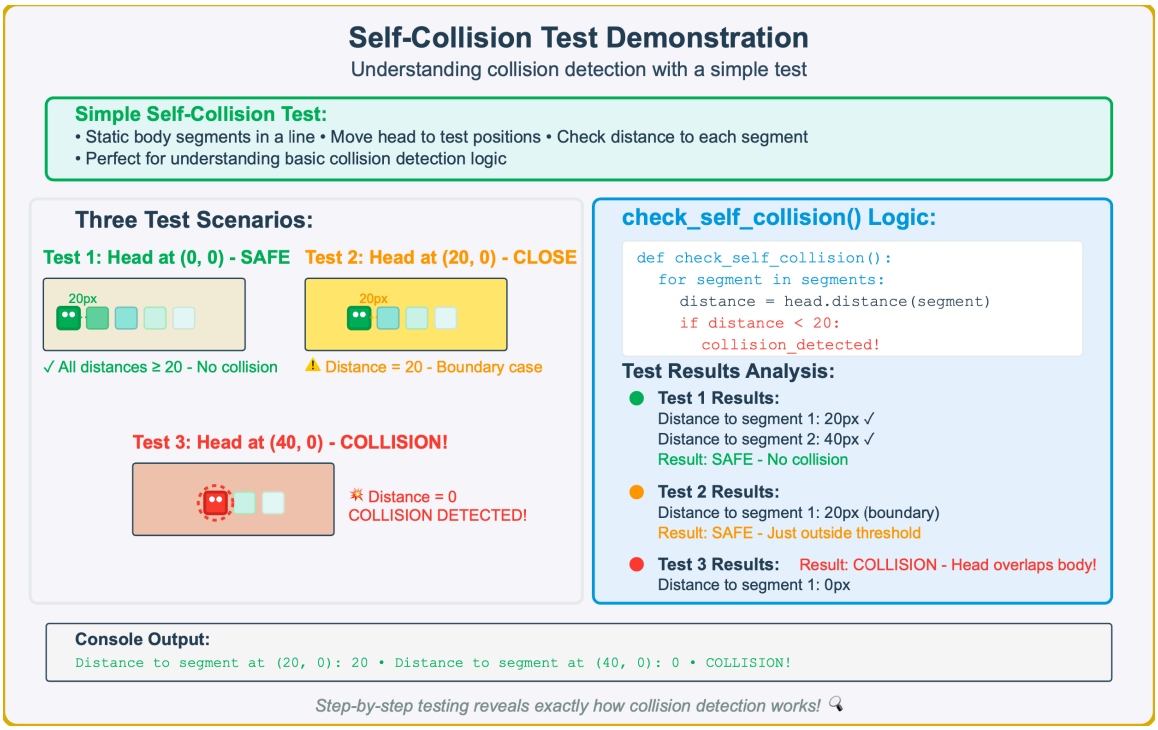

Figure 11.2 — Self—Collision Detection Test Scenarios. Three test cases demonstrate collision detection logic: (1) Head at (0,0) — SAFE with all distances ≥20 showing no collision, (2) Head at (20,0) — CLOSE with distance = 20 as a boundary case, (3) Head at (40,0) — COLLISION with distance = 0 detecting overlap. The check_self_collision() function loops through segments, calculates head.distance(segment), and triggers collision when distance < 20.

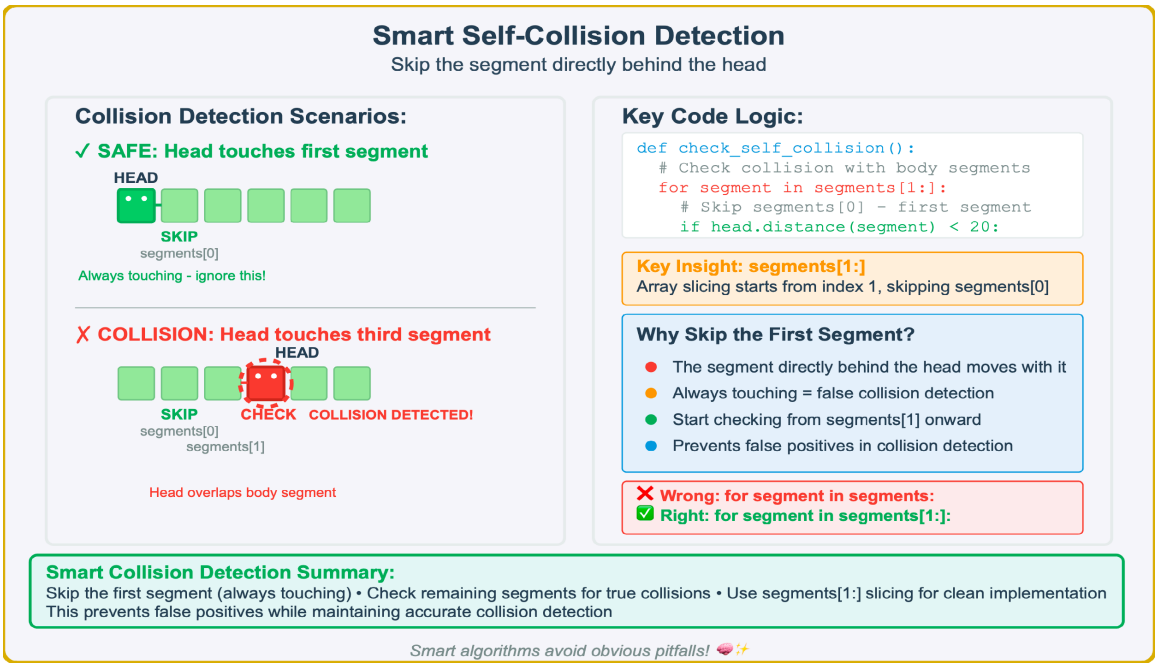

Figure 11.3 — Smart Self—Collision Detection — Skipping the First Segment. Two scenarios demonstrate why the first segment must be excluded: (1) SAFE — Head touches first segment (segments[0]) which is always touching and should be skipped, (2) COLLISION — Head touches third segment which represents a true collision. The code logic uses segments[1:] to skip the first segment, preventing false collision detection from the always—touching segment directly behind the head.

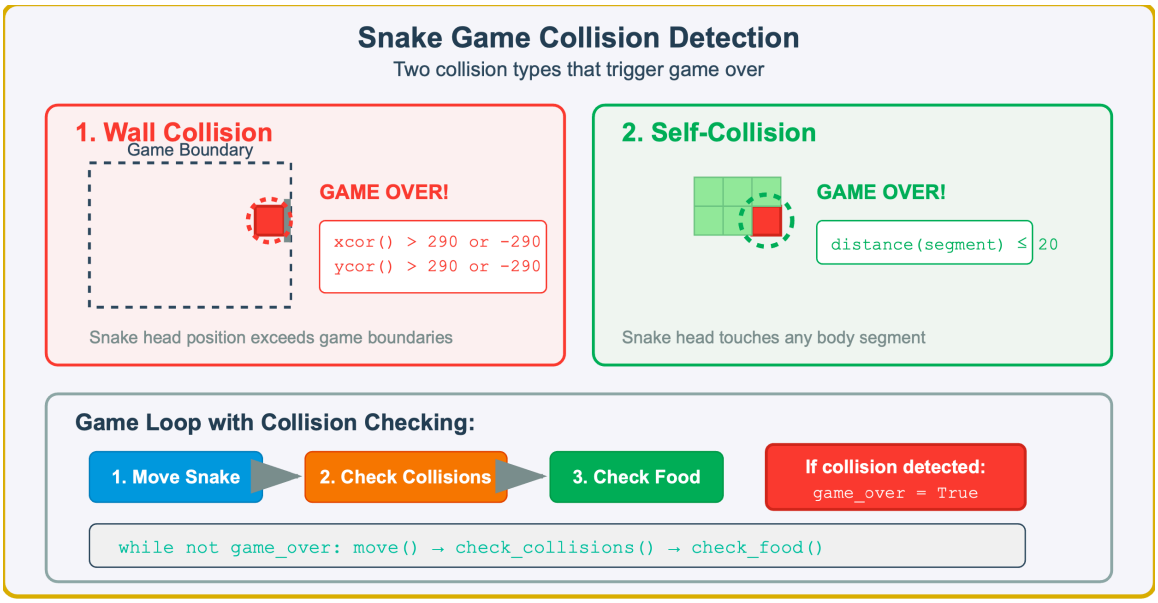

Figure 11.4 — Snake Game Collision Detection and Game Over Conditions. The game monitors two types of collisions: wall boundaries and self—collision, both triggering immediate game termination within the main game loop.

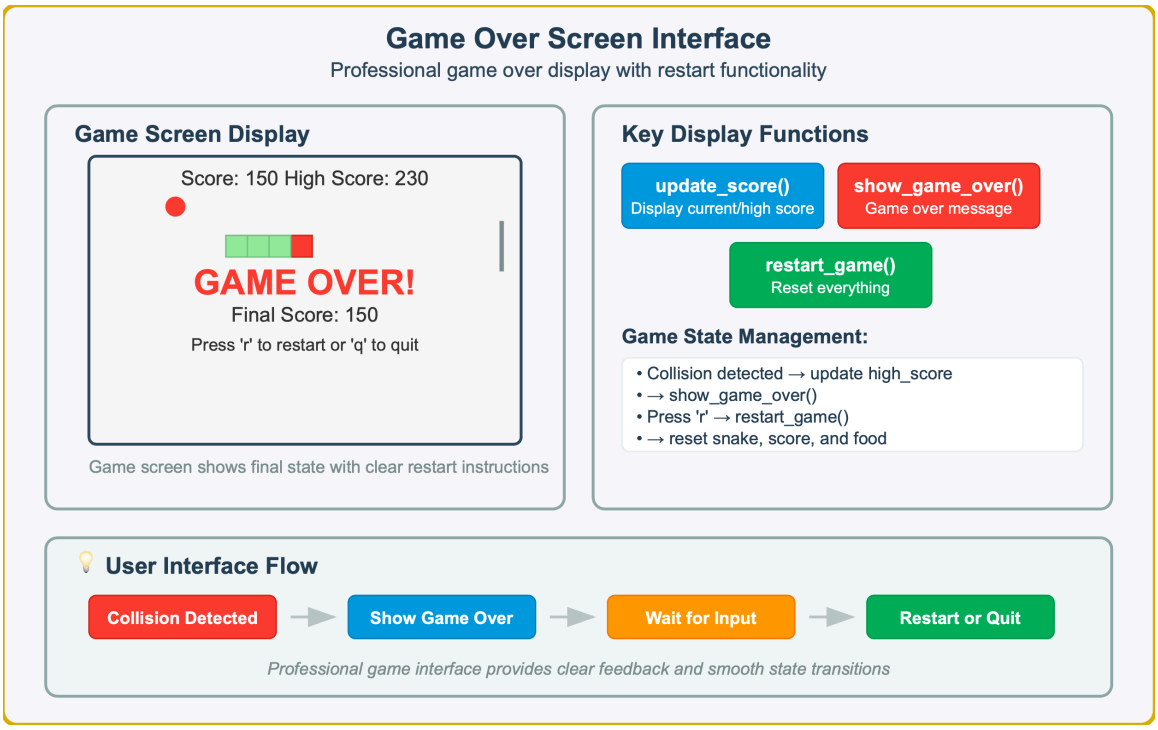

Figure 11.5 — Game Over Screen Interface and Restart Functionality. The game displays final score, restart instructions, and manages game state transitions through dedicated display functions and user input handling.

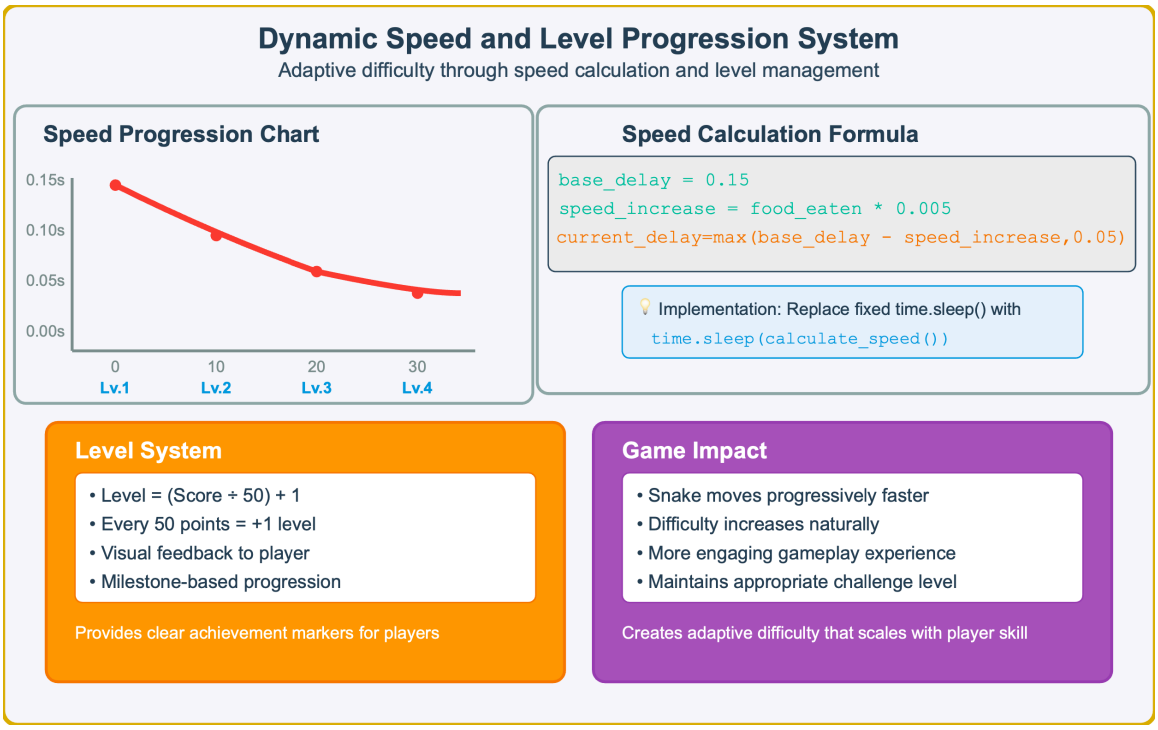

Figure 11.6 — Dynamic Speed and Level Progression System. The game calculates movement delay based on food eaten, with levels determined by score milestones, creating progressively challenging gameplay through adaptive speed increases.

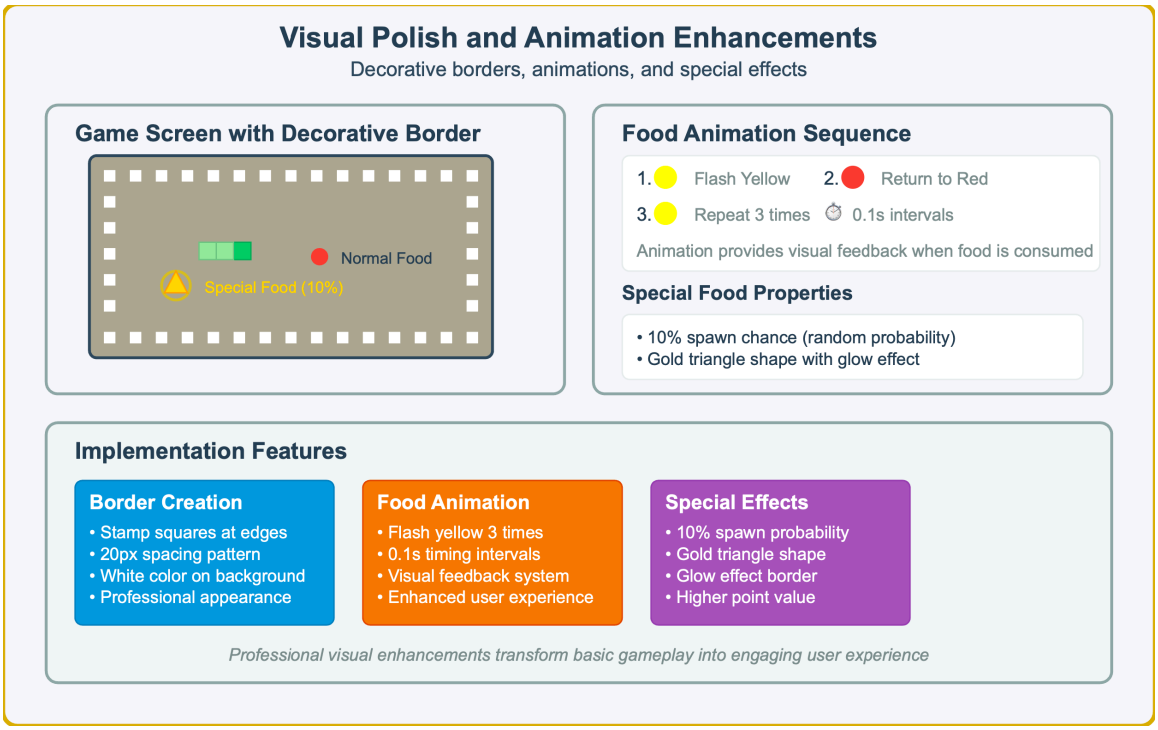

Figure 11.7 — Visual Polish and Animation Enhancements. The game features decorative border elements, animated food with yellow flash effects (0.1s intervals, 3 repetitions), and special golden food items with 10% spawn probability for enhanced visual appeal.

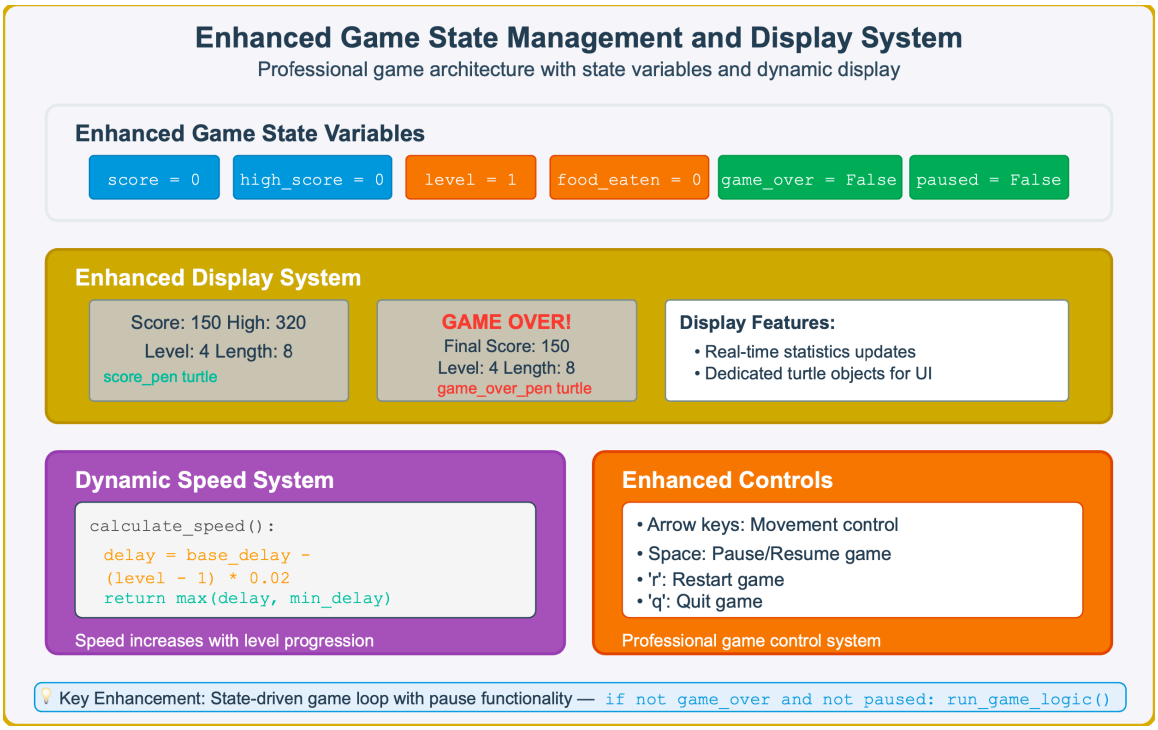

Figure 11.8 — Enhanced Game State Management and Display System. The game manages multiple state variables (score, high_score, level, food_eaten, game_over, paused) with dedicated display turtles for real—time statistics and enhanced controls including pause/resume functionality.

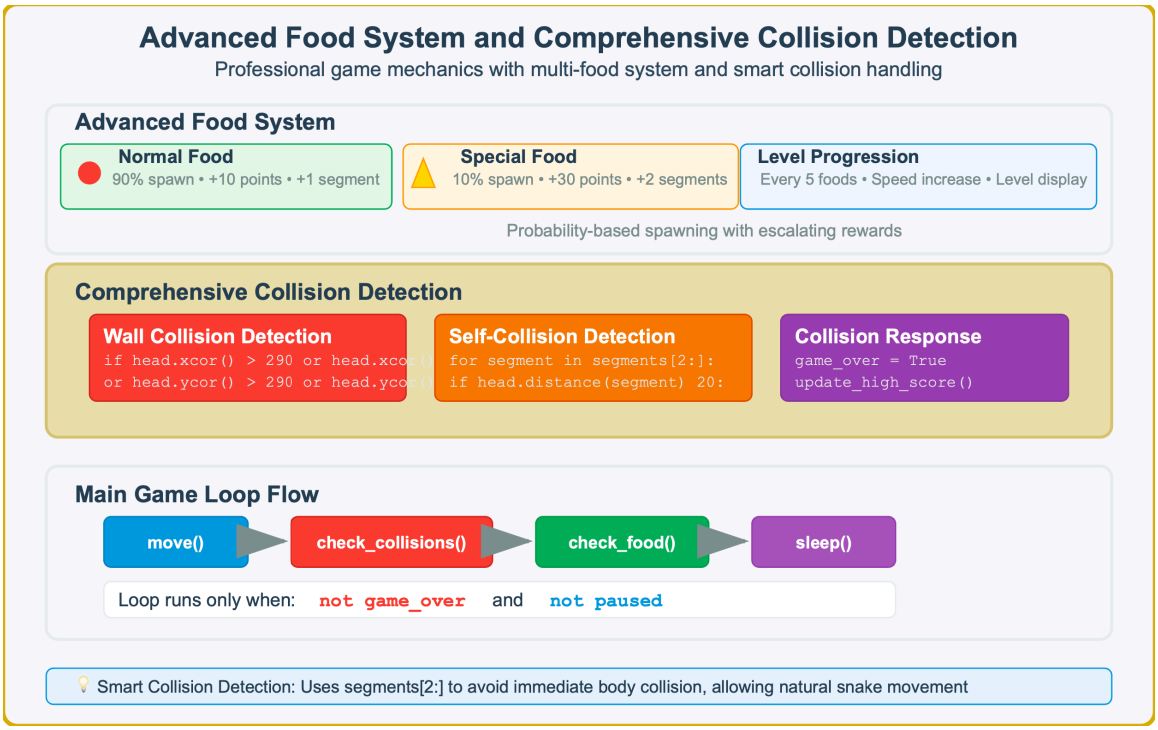

Figure 11.9 — Advanced Food System and Comprehensive Collision Detection. The game features normal food (90% spawn, +10 points, +1 segment) and special food (10% spawn, +30 points, +2 segments), with comprehensive collision detection for walls and self—collision, integrated into a state—driven game loop.

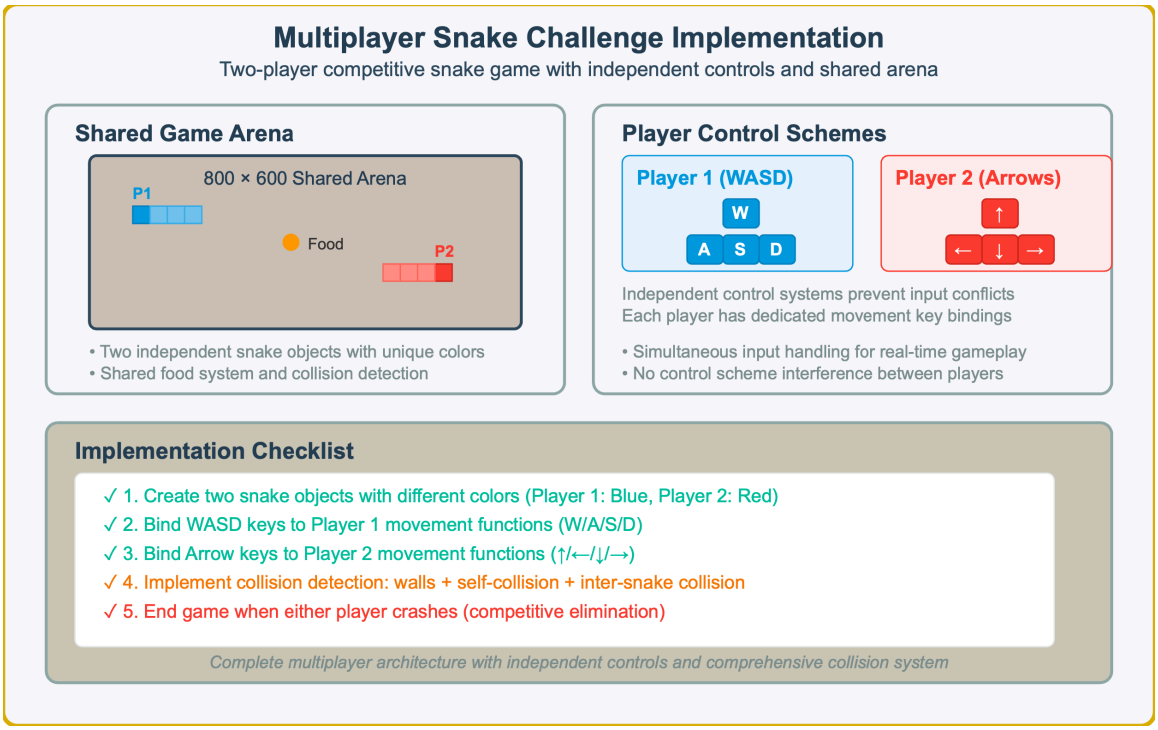

Figure 11.10 — Multiplayer Snake Challenge Implementation. The two—player game features independent snake objects with distinct control schemes (Player 1: WASD, Player 2: Arrow keys), shared arena gameplay, and comprehensive collision detection including walls, self—collision, and inter—snake collisions.

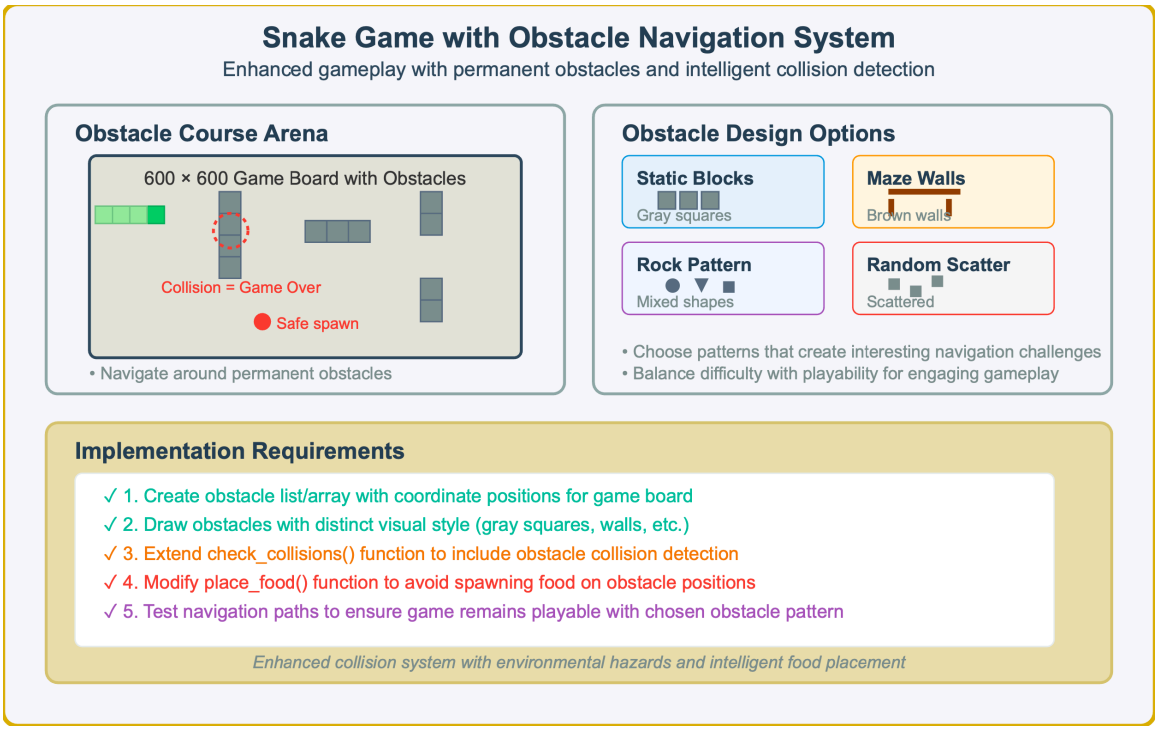

Figure 11.11 — Snake Game with Obstacle Navigation System. The enhanced game features permanent obstacles in a 600×600 arena with collision detection, safe food spawning that avoids obstacle positions, and multiple obstacle design options including static blocks, maze walls, rock patterns, and random placement.

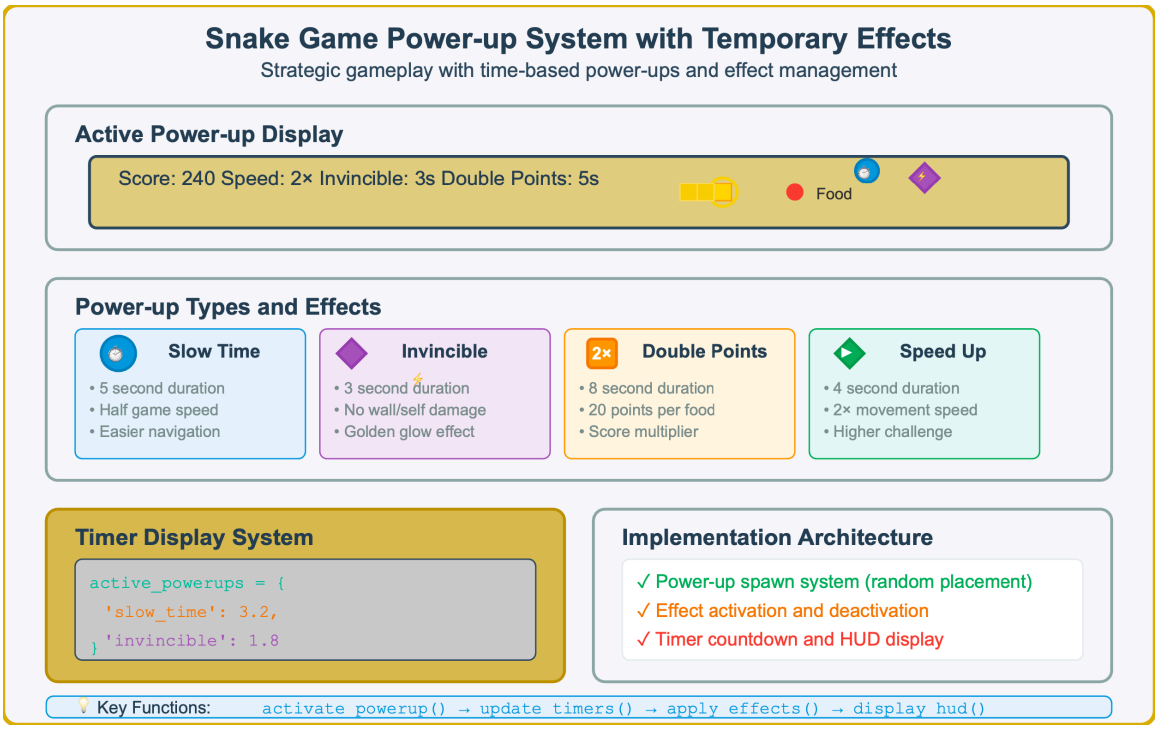

Figure 11.12 — Snake Game Power—up System with Temporary Effects. The enhanced game features four power—up types (Slow Time: 5s half—speed, Invincible: 3s no damage, Double Points: 8s 20pts per food, Speed Up: 4s 2× movement) with timer display system and strategic gameplay mechanics.